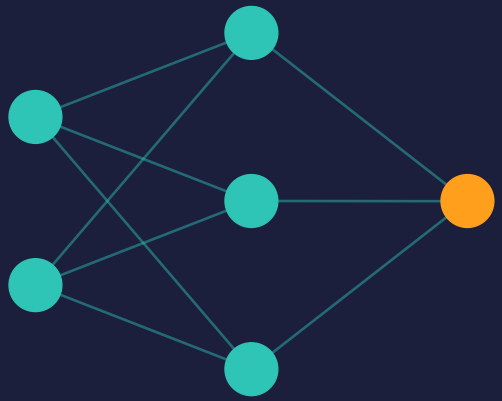
دوره یادگیری عمیق و شبکه‌های عصبی



جلسه ۷

شبکه‌های عصبی کانولوشنی (CNN)

کانولوشن، Padding و Pooling، Stride، میدان دید و اولین CNN

کیوان محمودی

در این جلسه

۱ چرا MLP برای تصویر مناسب نیست؟

۲ عملیات کانولوشن

۳ فیلترهای لبه‌یاب

۴ Stride، Padding و کانال‌ها

۵ Pooling و میدان دید

۶ اولین CNN روی Fashion-MNIST

۷ درون CNN: فیلترها و Feature Mapها

۸ تصاویر رنگی: CIFAR-10

سه مشکل MLP برای تصویر

۱ پارامتر زیاد

تصویر رنگی ۲۲۴×۲۲۴ بیش از ۱۵۰ هزار ورودی دارد؛ یک لایه ۱۰۰۰ نورونی حدود ۱۵۰ میلیون وزن می‌خواهد.

۲ از دست رفتن ساختار فضایی

با Flatten، پیکسل‌های همسایه دیگر همسایه نیستند.

۳ حساسیت به جابه‌جایی

الگویی که در گوشه بالا یاد گرفته شده، در گوشه پایین شناخته نمی‌شود.

دو ایده کلیدی CNN

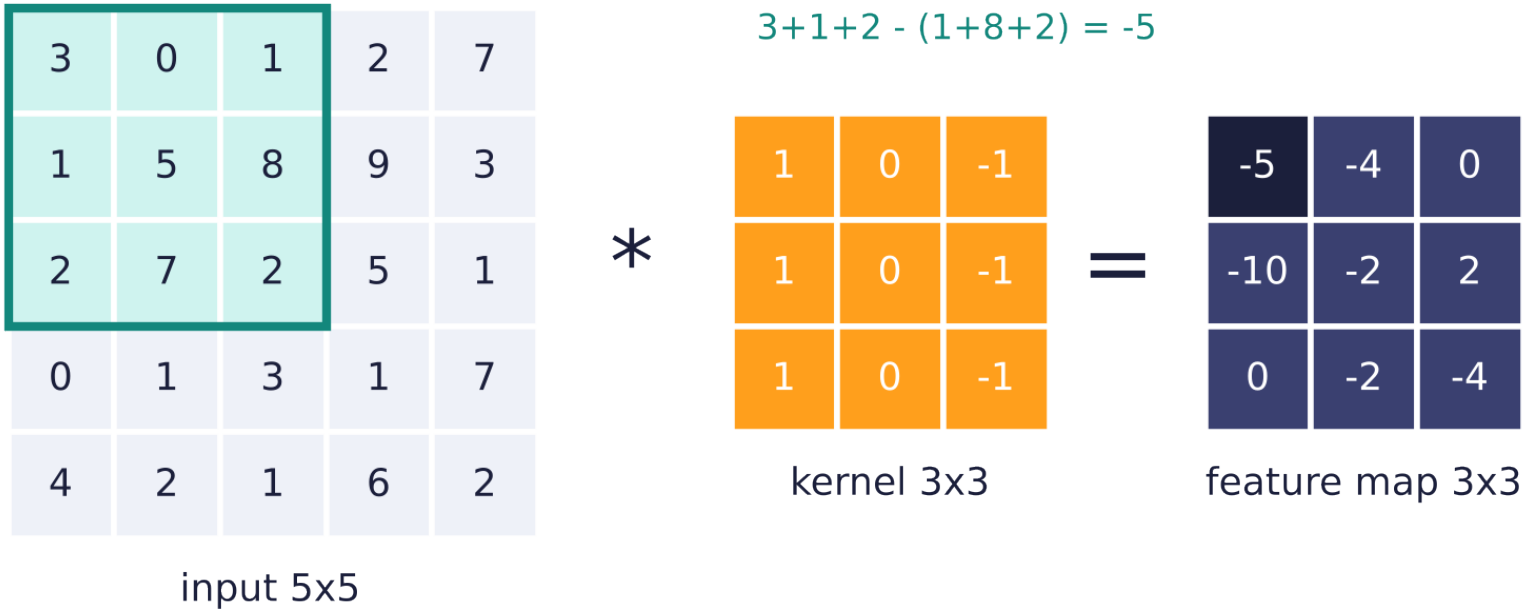
اتصال محلی (Local Connectivity)

- هر نورون فقط به یک ناحیه کوچک ورودی نگاه می‌کند.
- الگوهای محلی مثل لبه و بافت را می‌بیند.

اشتراک وزن (Weight Sharing)

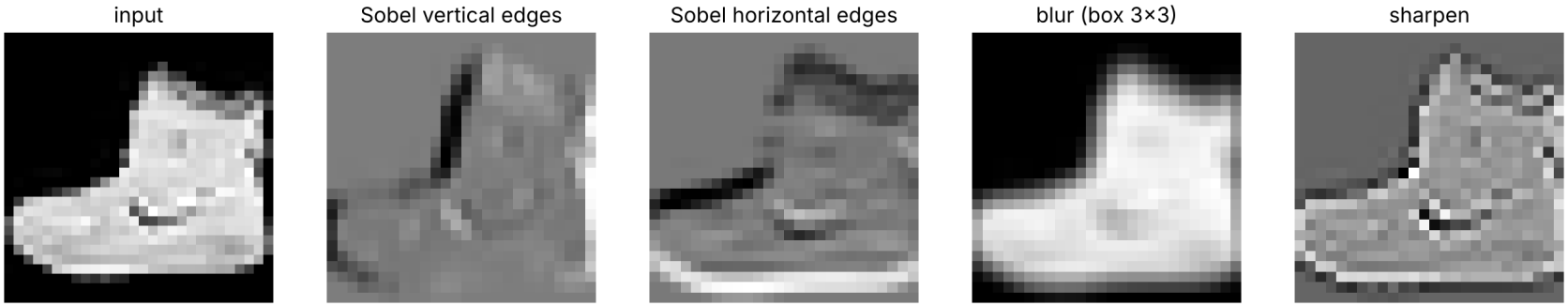
- یک فیلتر کوچک روی همه مکان‌های تصویر می‌لغزد.
- الگو در هر جای تصویر شناخته می‌شود.
- تعداد پارامتر مستقل از اندازه تصویر است.

عملیات کانولوشن



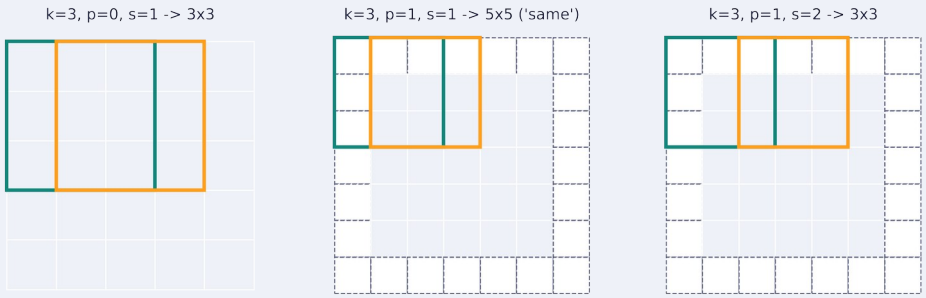
کرنل روی تصویر می‌لغزد؛ در هر مکان حاصل‌ضرب عنصربه‌عنصر جمع زده می‌شود و یک عدد از Feature Map ساخته می‌شود

فیلترهای کلاسیک روی یک تصویر واقعی



در CNN مقادیر کرنل‌ها پارامترهای قابل آموزش‌اند و شبکه خودش فیلترهای مفید را یاد می‌گیرد

Padding، Stride و ابعاد خروجی



- Padding: افزودن صفر دور تصویر
- با کرنل ۳×۳ و padding=1 ابعاد حفظ می‌شود.
- Stride: فاصله گام‌های لغزش؛ با stride=2 ابعاد نصف می‌شود.
- هر فیلتر روی همه کانال‌های ورودی اعمال و جمع می‌شود.

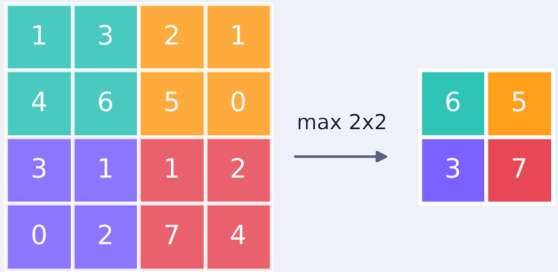
دو فرمول که باید حفظ باشید

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1$$

$$\#\text{params} = C_{\text{out}} \times (C_{\text{in}} \times k \times k + 1)$$

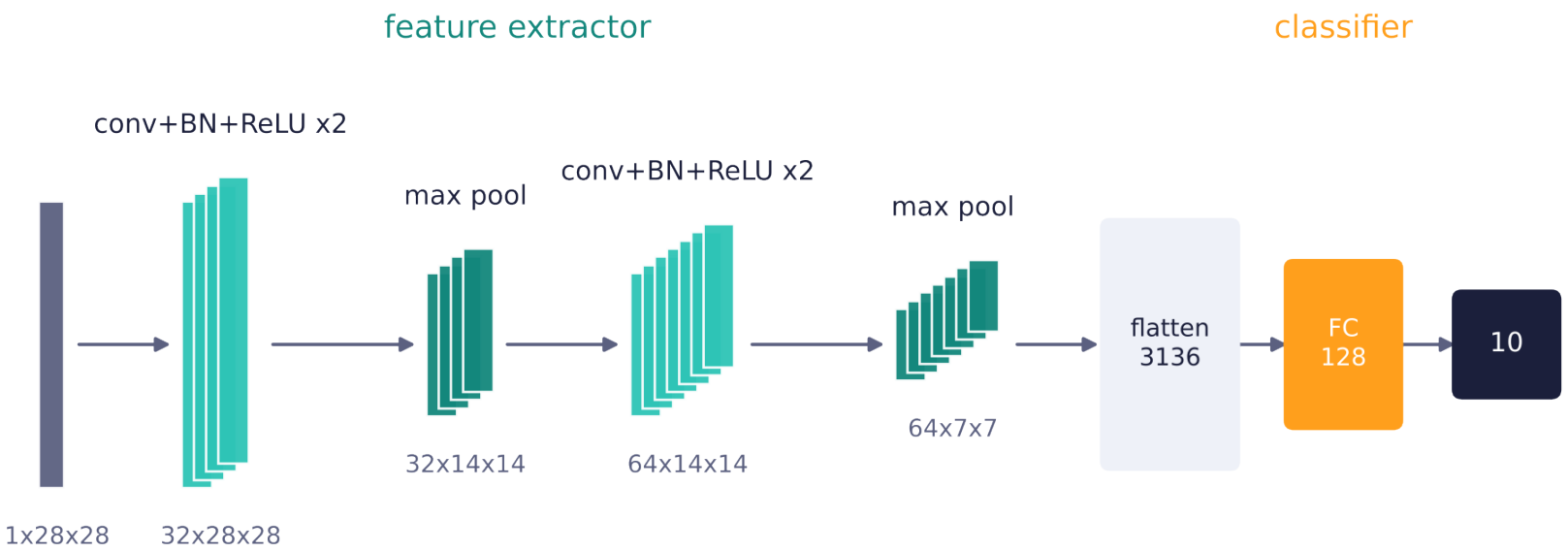
- ۱۶ فیلتر ۳×۳ روی تصویر رنگی: فقط ۴۴۸ پارامتر
- لایه Linear معادل همین ورودی و خروجی: بیش از ۵۰ میلیون پارامتر
- nn.Conv2d ورودی با شکل (N, C, H, W) می‌گیرد.

Pooling و میدان دید (Receptive Field)



- Max Pooling: بیشینه هر ناحیه ۲×۲
- Global Average Pooling در CNNهای مدرن به‌جای Flatten
- دو لایه ۳×۳ پشت سر هم: میدان دید ۵×۵
- یادگیری سلسله‌مراتبی: لبه ← جزء ← شیء

معماری CNN جلسه: دو بلوک کانولوشنی و طبقه‌بند



۴۶۸,۲۰۲ پارامتر؛ لایه‌های کانولوشنی فقط حدود ۶۵ هزار و بقیه در لایه Linear اول

پیاده‌سازی در PyTorch

```
def conv_block(c_in, c_out):
    return nn.Sequential(
        nn.Conv2d(c_in, c_out, 3, padding=1),
        nn.BatchNorm2d(c_out), nn.ReLU(),
        nn.Conv2d(c_out, c_out, 3, padding=1),
        nn.BatchNorm2d(c_out), nn.ReLU(),
        nn.MaxPool2d(2))

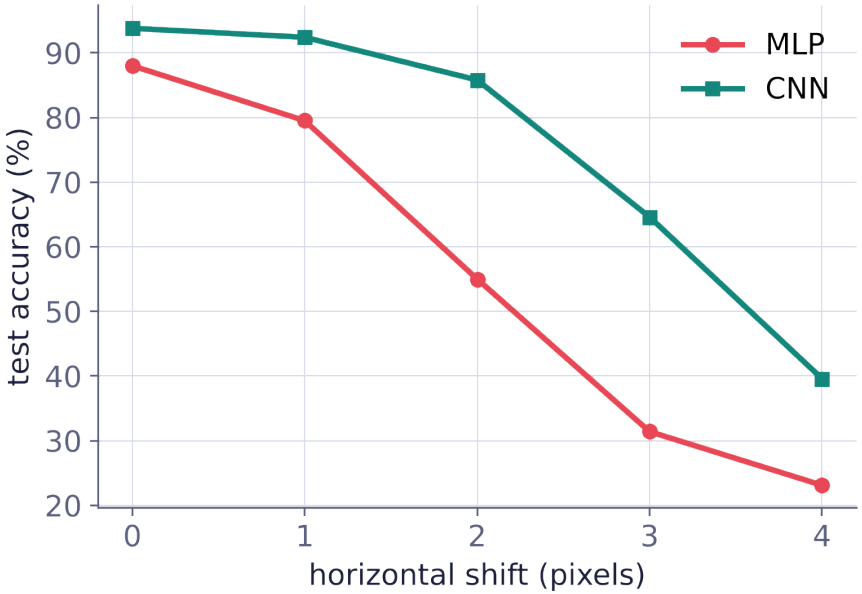
class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.features = nn.Sequential(conv_block(1, 32),
                                      conv_block(32, 64))
        self.classifier = nn.Sequential(
            nn.Flatten(), nn.Dropout(0.3),
            nn.Linear(64 * 7 * 7, 128), nn.ReLU(),
            nn.Linear(128, 10))

    def forward(self, x):
        return self.classifier(self.features(x))
```

- بلوک: Conv → BatchNorm → ReLU → Pool
- تعداد کانال‌ها زیاد و ابعاد مکانی کم می‌شود.
- آموزش: AdamW و OneCycleLR، ۱۰ Epoch

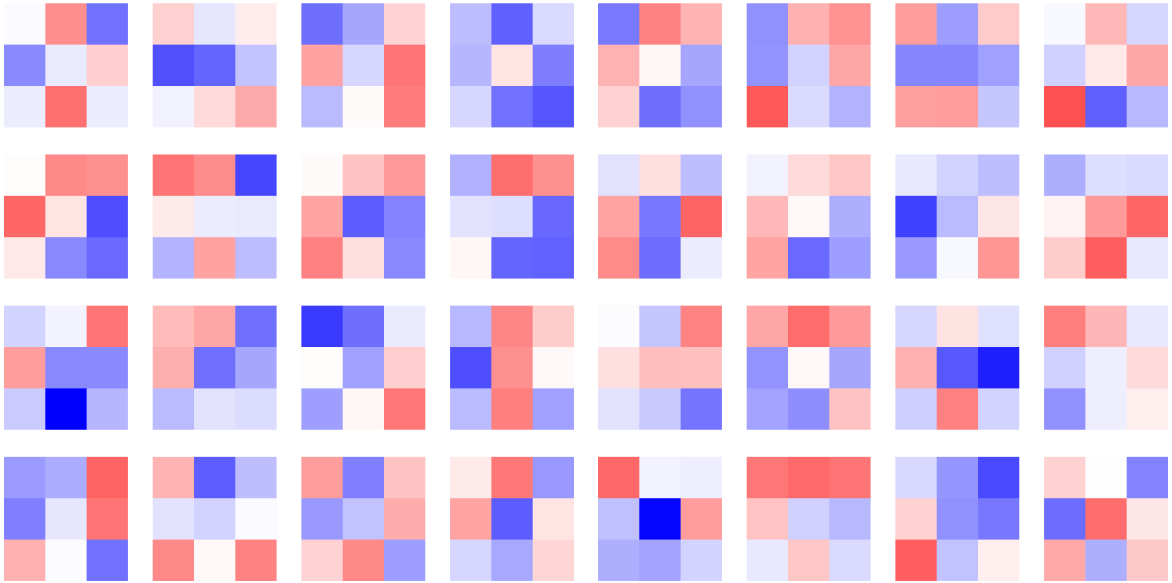
نتیجه: CNN در برابر MLP

- دقت آزمون CNN: ۹۳/۸٪
- MLP جلسه ۴: ۸۸/۱٪
- خطاهای بین T-shirt، Shirt و Coat به‌طور محسوسی کم شد.
- با جابه‌جایی ۴ پیکسلی: CNN ۴۰٪ در برابر MLP ۲۳٪



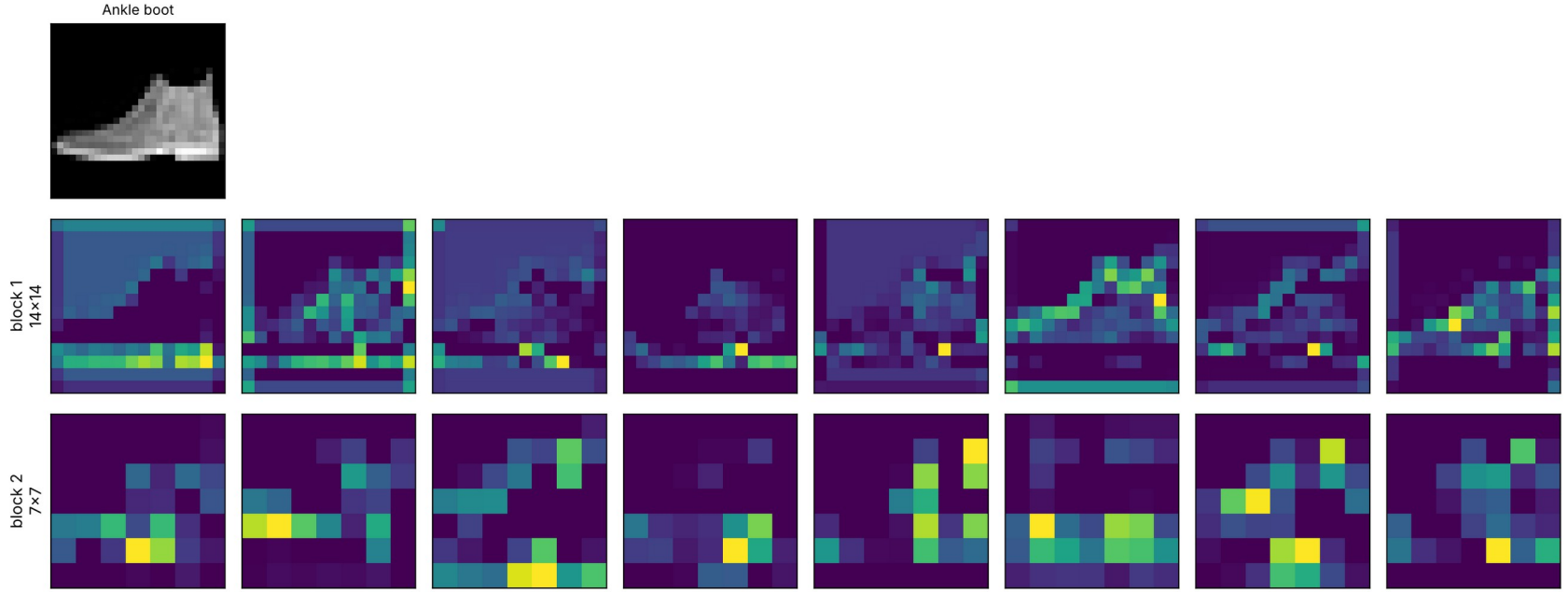
فیلترهای یادگرفته‌شده در لایه اول

first-layer 3×3 filters (red = positive, blue = negative)



آشکارسازهای لبه و بافت در جهت‌های مختلف؛ هیچ‌کدام دستی طراحی نشده‌اند

Feature Mapها: از لبه تا مفهوم



بلوک اول هنوز شبیه تصویر است؛ بلوک دوم انتزاعی‌تر و هر نقشه به یک ویژگی سطح بالاتر حساس است

تصاویر رنگی: CIFAR-10

- ۶۰,۰۰۰ تصویر رنگی ۳۲×۳۲ در ۱۰ کلاس
- بسیار سخت‌تر از Fashion-MNIST
- سه بلوک کانولوشنی، Global Average Pooling و Augmentation
- این بخش را روی GPU در Colab اجرا کنید.
- برای دقت بالاتر به شبکه‌های عمیق‌تر نیاز داریم: جلسه ۸

جمع‌بندی

- CNN با اتصال محلی و اشتراک وزن، پارامترها را کم و ساختار فضایی را حفظ می‌کند.
- کانولوشن = لغزاندن فیلتر یادگرفتنی؛ هر فیلتر یک Feature Map می‌سازد.
- دو فرمول کلیدی: ابعاد خروجی و تعداد پارامتر لایه کانولوشنی
- Pooling ابعاد را کم و میدان دید را زیاد می‌کند.
- ساختار رایج: بلوک‌های Conv → BN → ReLU → Pool و سپس طبقه‌بند
- CNN از MLP دقیق‌تر و نسبت به جابه‌جایی مقاوم‌تر است.

تمرین

۱ تابع conv2d_naive را با stride و padding گسترش دهید و با F.conv2d مقایسه کنید.

۲ ابعاد خروجی و تعداد پارامترهای هر لایه CifarCNN را دستی حساب کنید.

۳ MaxPool2d را با کانولوشن stride=2 جایگزین کنید؛ پارامتر و دقت چه تغییری می‌کند؟

۴ BatchNorm را از CifarCNN حذف کنید و منحنی‌های آموزش را مقایسه کنید.

۵ Confusion Matrix مدل CIFAR-10 را رسم و خطاها را تحلیل کنید.